Architecting a Distributed, Hybrid Linux Operating System: Multi-Ecosystem Toolchain Integration, Thermal-Aware Kernel Topologies, and Decentralized CPU Pooling

The engineering of a custom Linux distribution that seamlessly bridges the divide between a stable server environment, a specialized penetration testing workstation, and a distributed compute cluster orchestrator represents a formidable architectural challenge. Historically, operating systems have been tailored for highly specific, mutually exclusive use cases. Distributions like Debian prioritize long-term stability and strict dependency resolution, making them ideal for servers. Conversely, security-focused derivatives like Kali Linux and Parrot OS employ rolling-release models designed to rapidly deliver the most cutting-edge security toolchains, often at the expense of base system stability. Attempting to merge these ecosystems typically results in catastrophic dependency collapse.

Concurrently, the hardware profile of a laptop acting as a localized server introduces severe thermal and power management constraints that traditional server kernels are not optimized to handle. Laptops are thermally constrained devices engineered for burst workloads, whereas servers require sustained throughput. Finally, the ambition to utilize a local area network (LAN) as a decentralized pool of CPU resources necessitates the integration of sophisticated High-Throughput Computing (HTC) middleware directly into the operating system's base image, circumventing the obsolete paradigms of kernel-level process migration.

This report provides an exhaustive, expert-level architectural blueprint for constructing this highly specialized Linux distribution. The analysis proceeds through four foundational pillars: the synthesis of the base operating system using Debian live-build infrastructure; the integration of third-party security repositories (Kali, Parrot) via advanced cryptographic pinning and meta-distribution hijacking techniques; the configuration of a custom Linux kernel optimized for hybrid laptop/server preemption and thermal dynamics; and the implementation of a decentralized, transparent compute cluster for CPU cycle scavenging across a local network.

Operating System Synthesis via Debian Live-Build

The foundation of any highly customized, reproducible Linux distribution relies on robust bootstrapping and image generation frameworks. For a system rooted in Debian, the optimal approach eschews manual post-installation scripts in favor of the live-build and debootstrap toolchains.[1] These tools permit the programmatic definition of the entire operating system,

from the initial chroot environment to the final bootable ISO hybrid image.

The Debootstrap and Live-Build Architecture

The debootstrap utility operates by constructing a minimal Debian base system within a subdirectory of a host operating system, requiring no installation media other than access to a Debian package repository.[4] While debootstrap alone is sufficient for generating root filesystems for containers or chroot jails, the creation of a distributable, bootable ISO requires the live-build suite.[5]

The live-build process is controlled via a highly structured configuration directory initialized using the lb config command.[6] This directory tree dictates every aspect of the final distribution, allowing engineers to programmatically inject packages, repositories, and custom binaries before the root filesystem is compressed into a SquashFS payload.[7] The configuration hierarchy is strictly defined:

- config/package-lists/: Defines the exact .list.chroot files containing the names of packages to be retrieved and installed during the build phase.[6]
- config/archives/: Houses third-party repository definitions and their corresponding cryptographic keys, ensuring that external package sources are available within the chroot environment.[9]
- config/includes.chroot/: Allows the injection of raw files, scripts, and configuration payloads directly into the root filesystem of the target distribution before it is compressed.[2]
- config/hooks/live/ and config/hooks/normal/: Contain executable scripts that run within the chroot environment during the final stages of the build, permitting deep system modifications such as user creation, default service enabling, or specific file permission alterations.[10]

By utilizing this infrastructure, the distribution can be engineered to function in a dual-mode capacity: booting as an ephemeral live system for immediate use or executing a deployment sequence via a graphical installer (such as Calamares) to permanently write the system to local disk storage.[10]

Custom Kernel Integration within the Build Pipeline

Because this distribution requires a highly specialized Linux kernel tailored for a hybrid laptop-server environment, the default kernel provided by the Debian repositories must be overridden during the live-build execution. The compilation of a custom kernel typically involves configuring the source via make menuconfig, tweaking the parameters, and executing a build process to generate installable Debian packages.[12] Modern kernel compilation utilizes the make bindeb-pkg target (superseding the deprecated make-kpkg utility) to output linux-image, linux-headers, and linux-libc-dev .deb archives.[13]

Integrating these newly compiled packages into the live-build process requires placing the .deb

files directly into the config/packages.chroot/ directory.[15] To ensure the build system prioritizes the custom kernel over the upstream defaults, the --linux-packages argument must be passed to the lb config command, explicitly defining the custom kernel stub.[9]

Furthermore, because custom kernels often require specific module loading parameters to boot successfully on diverse hardware, the initial RAM disk (initramfs) must be properly regenerated within the build environment. This is accomplished by placing an executable shell script within the config/hooks/normal/ directory.[16] This hook script instructs the system to execute update-initramfs -c -k all after the custom kernel packages are unpacked, ensuring that the final SquashFS payload contains a fully functional boot chain tailored to the custom kernel parameters.[16] Without this hook, the live system would likely encounter kernel panics during the boot sequence due to missing block storage or cryptographic modules.

Cross-Ecosystem Package Management: Resolving the FrankenDebian Dilemma

A core requirement of this distribution is the native execution of security and penetration testing tools typically exclusive to distributions like Kali Linux and Parrot OS. Because Kali and Parrot are direct derivatives of Debian, they share the Advanced Package Tool (APT) architecture. However, directly appending their repositories to a Debian Stable installation creates a highly unstable state colloquially known in the engineering community as a "FrankenDebian".[17]

The Mechanics of Dependency Collapse

Debian Stable relies on a carefully orchestrated web of dependencies, most notably the GNU C Library (libc6), which dictates system-wide Application Programming Interface (API) and Application Binary Interface (ABI) compatibility. Kali Linux and Parrot OS utilize a "rolling release" model based on Debian Testing/Unstable, meaning their core libraries are significantly newer than those in Debian Stable.[19]

If an administrator adds the Kali repository to a Debian Stable system and issues an apt update && apt upgrade command, the package manager will observe the higher version numbers in the Kali repository and attempt to upgrade foundational system libraries to satisfy the dependency requirements of the new tools.[21] This cascades into a catastrophic conflict where applications compiled against older libraries fail to link dynamically, rendering the graphical desktop environment, essential system utilities, and even the bootloader completely inoperable.[21]

To safely access the expansive toolchains of Kali and Parrot without jeopardizing the stability of the Debian server core, the architecture must employ one of two methodologies: advanced cryptographic APT pinning, or meta-distribution integration via Bedrock Linux.

Methodology A: Cryptographic Verification and APT Pinning

The first approach involves strict manipulation of the APT dependency resolver combined with isolated cryptographic keyrings. The traditional method of importing repository keys via apt-key add has been deprecated due to severe security vulnerabilities; adding a key to the global /etc/apt/trusted.gpg keyring forces the system to trust that key for all configured repositories, opening vectors for cross-repository spoofing and supply chain attacks.[23]

Modern Debian repository integration dictates that OpenPGP certificates must be securely downloaded over HTTPS, de-armored into a binary format, and placed in a dedicated, root-writable directory such as /usr/share/keyrings/.[18] For the Kali repository, the sequence requires extracting the key directly into an isolated file:

curl https://archive.kali.org/archive-key.asc | gpg --dearmor > /usr/share/keyrings/kali-archive-keyring.gpg.[23]

Following the acquisition of the cryptographic key, the repository source definition must explicitly reference this isolated keyring using the signed-by parameter within the /etc/apt/sources.list.d/ directory. This is optimally formatted using the modern DEB822 multiline format to ensure absolute clarity:

Types: deb

URIs: http://http.kali.org/kali

Suites: kali-rolling

Components: main non-free contrib

Signed-By: /usr/share/keyrings/kali-archive-keyring.gpg

This configuration isolates the trust boundary, ensuring that only packages originating from the Kali servers are authenticated by the Kali key, completely eliminating global key-spoofing risks.[23]

With the repository safely added, dependency collapse is prevented by manipulating the internal logic of the APT dependency resolver through APT pinning.[21] By default, APT assigns a priority of 500 to all packages in standard repositories. When two versions of the same package exist across different repositories, APT selects the highest version number. To shield the core Debian OS, a preference file must be created at /etc/apt/preferences.d/99kali dictating a severely degraded priority for the entire Kali repository.[23]

Priority Level	**APT Behavior Mapping**
1000+	Downgrading allowed; enforces installation even if a higher version is installed locally.

500 - 990	Standard priority; higher versions will upgrade lower versions seamlessly.
100 - 499	Will install unless a newer version is available locally or in another repository.
1 - 99	Will *never* automatically install or upgrade. Must be explicitly requested by the user.
< 0	Prevents the package from being installed entirely.

By assigning Pin-Priority: 1 or 50 to the Kali origin, the system is instructed to acknowledge the existence of the packages without ever preferring them over the base system's packages during routine upgrades.[23]

Package: *

Pin: release o=Kali

Pin-Priority: 1

Under this architecture, when the user wishes to install a specific penetration testing tool, they must explicitly invoke the target release via the command line: apt install -t kali-rolling <package>.[21] The package manager will retrieve the requested tool and its immediate, isolated dependencies from the Kali repository, leaving the overarching libc6 and core Debian infrastructure completely untouched.[21]

Methodology B: Meta-Distribution Integration via Bedrock Linux

While APT pinning is functional, it remains fragile; deep dependency chains in complex penetration testing tools can still occasionally trigger conflicts. A more robust, architecturally sound solution is the integration of Bedrock Linux. Bedrock Linux is a "meta Linux distribution" that allows users to mix and match components from typically incompatible distributions by integrating them into a largely cohesive system.[27]

Instead of manipulating a single package manager, Bedrock Linux introduces the concept of "strata." A stratum is a collection of files associated with a specific distribution (e.g., a Debian stratum, a Kali stratum, a Parrot stratum).[29] Bedrock utilizes advanced filesystem mounts

(specifically etcfs) and chroot-like isolation to ensure that executables from one stratum resolve their dependencies against libraries within their own stratum, while still seamlessly interacting with the global filesystem, user home directories, and process space.[27]

Integrating Bedrock Linux into the custom Debian live-build ISO requires the execution of a "hijack" script during the build's hook phase.[31] The bedrock-linux-install.sh --hijack command converts the base Debian installation into a Bedrock system in-place.[32] Once the hijack is complete, the brl fetch utility can be programmatically invoked to download and bootstrap full environments for Kali and Parrot.[27]

By defining the Debian base as the primary rootfs stratum and Kali/Parrot as secondary strata, the system gains the unparalleled ability to execute tools from any distribution transparently. A user can run an Apache server managed by Debian's systemd, while concurrently executing metasploit from the Kali stratum and anonsurf from the Parrot stratum, all within the same terminal window without a single dependency conflict.[27] This approach perfectly satisfies the user's requirement to seamlessly fetch and install tools from diverse security distributions onto their custom OS.

Custom Kernel Engineering: The Hybrid Laptop-Server Topologies

Deploying a laptop to act as a local server presents severe hardware and thermal management contradictions. Laptops are heavily constrained physical environments engineered for burst-performance workloads and aggressive thermal throttling to preserve skin temperature and battery life.[34] Conversely, enterprise servers demand sustained, high-throughput execution with minimal latency over extended periods. Modifying the Linux kernel configuration to navigate this dichotomy requires intricate tuning across three specific domains: execution preemption, thermal-aware P-state power governance, and network stack buffering.

Preemption Models and CONFIG_PREEMPT_DYNAMIC

The Linux kernel's preemption model dictates how and when tasks are scheduled, interrupted, and prioritized. Traditionally, kernels are compiled with a static preemption model selected during the build phase. A PREEMPT_NONE configuration maximizes raw computational throughput by preventing the kernel from interrupting running user-space tasks to handle other events.[36] This minimizes context-switching overhead and is the standard for high-performance servers, data processing nodes, and computational clusters. Conversely, a PREEMPT_FULL configuration allows the kernel to preempt almost any task, drastically lowering latency to ensure highly responsive desktop environments and real-time audio/video processing.[36]

Because the intended distribution will serve simultaneously as a daily-driver desktop laptop and a continuous backend compute server, statically defining preemption forces an unacceptable compromise. Hardcoding PREEMPT_NONE would render the laptop's graphical

interface sluggish and unresponsive, while hardcoding PREEMPT_FULL would degrade the computational throughput required for the local server workloads.

The architectural solution lies in the implementation of CONFIG_PREEMPT_DYNAMIC.[39] Introduced in recent kernel lineages (post-5.12), this feature leverages static calls and objtool capabilities to allow the preemption model to be altered dynamically at boot time via kernel command-line parameters, without necessitating a complete kernel recompile.[39]

Kernel Parameter	Preemption Model	Optimal Use Case	Target Workload Profile
preempt=none	PREEMPT_NONE	Dedicated Server Node	High throughput, batch processing, minimal context switching.
preempt=voluntary	PREEMPT_VOLUNTARY	Balanced System	General purpose, slight latency reduction with good throughput.
preempt=full	PREEMPT_FULL	Desktop / Interactive	Low latency, highly responsive GUI, real-time multimedia.

By enabling CONFIG_PREEMPT_DYNAMIC=y alongside HAVE_STATIC_CALL_INLINE in the custom kernel configuration, the user gains unprecedented operational flexibility.[39] When the laptop is docked and functioning purely as a cluster master node, the user can append preempt=none to the GRUB bootloader parameters to maximize compute efficiency. When the laptop is used for interactive pentesting or desktop tasks, a reboot with preempt=full restores the required responsiveness.[37]

Similarly, the timer interrupt frequency (CONFIG_HZ) must be considered. While servers generally utilize CONFIG_HZ=250 or 100 to reduce interrupt overhead, a hybrid system benefits from CONFIG_HZ=1000 to ensure smooth interactive performance, relying on dynamic ticks (CONFIG_NO_HZ_IDLE) to allow the CPU to sleep when unburdened.[36]

Advanced Thermal and P-State Governance

Because laptops possess highly restrictive cooling architectures, running sustained server or

cluster workloads will rapidly induce thermal throttling. Under standard configurations, when a laptop processor hits a thermal threshold, the hardware aggressively downclocks the CPU to prevent physical damage, leading to massive, unpredictable latency spikes and computational bottlenecks.[43] Standard ACPI cpufreq governors (such as ondemand or conservative) are legacy software implementations that are generally too slow to react optimally to the micro-fluctuations in localized server workloads.

For modern x86 architectures, the kernel must be configured to utilize hardware-managed P-state drivers (intel_pstate or amd_pstate).[44] These drivers bypass traditional OS-level frequency scaling, interacting directly with the hardware's internal microcontrollers. By utilizing APERF (Actual Performance) and MPERF (Maximum Performance) Model-Specific Registers (MSRs), the hardware autonomously calculates CPU utilization and adjusts the clock frequency and voltage at the microsecond level.[44] This hardware-level scaling (Hardware P-States or HWP) provides vastly superior energy efficiency and allows for sustained performance profiles without triggering sudden thermal limits.[44]

Furthermore, to manage thermal limits proactively rather than reactively, the custom distribution must integrate thermald into the userspace.[43] Unlike reactive hardware throttling, thermald is a Linux daemon developed specifically to monitor CPU digital temperature sensors and apply compensation before the hardware takes aggressive, system-halting correction actions.[43] By tapping into advanced kernel drivers such as the Running Average Power Limit (RAPL) and Intel PowerClamp, thermald gracefully injects idle cycles into the CPU pipeline.[43] This cools the processor smoothly, ensuring that the laptop delivers a consistent, predictable computational output suitable for server tasks, rather than oscillating violently between maximum turbo frequencies and thermal throttling minimums.

Network Stack Buffering for Cluster Throughput

As a master node orchestrating a local cluster, the laptop will act as the primary ingress and egress point for high volumes of inter-process communication, task distribution, and data sharing over the LAN. Default Linux network stack settings are highly conservative to preserve RAM, which frequently leads to packet drops when a server is flooded with simultaneous communication from dozens of worker nodes.[47]

The kernel configuration must be modified to account for high-throughput Ethernet interfaces by increasing the Receive (RX) and Transmit (TX) ring buffer sizes. These circular hardware buffers hold incoming packets until the CPU's software interrupts (SoftIRQs) can drain them into the sk_buff kernel structures.[47] Enlarging these buffers via ethtool or through specific sysctl configurations (e.g., net.core.somaxconn, net.core.rmem_max, net.core.wmem_max) prevents dropped packets during synchronized cluster reporting phases.[47]

Additionally, compiling the kernel with support for eXpress Data Path (XDP) alongside the BPF (Berkeley Packet Filter) subsystem enables the ability to bypass standard kernel networking stacks for specific cluster applications in the future.[49] XDP allows user-space cluster

applications to process incoming packets directly from the NIC driver's receive ring before the kernel allocates memory for them.[49] For a master node attempting to synchronize computational tasks across a LAN, this zero-copy packet processing dramatically reduces CPU overhead and network latency.[50]

Distributed Computing: CPU Cycle Scavenging and Resource Pooling

The final architectural mandate is the integration of software capable of chaining multiple LAN-connected computers into a unified CPU resource pool. The user's query describes a specific topology: a decentralized environment where arbitrary, secondary computers connected to a router or switch act as transparent processing extensions, allowing the master laptop to scavenge their unused CPU cycles for additional compute power.

The Obsolescence of the Single System Image (SSI)

Historically, the exact requirement described by the user—making a cluster of machines appear and function as a single, massive computer—was fulfilled by Single System Image (SSI) clustering technologies such as MOSIX, OpenMosix, OpenSSI, and Kerrighed.[51] These systems operated deep within the Linux kernel to create a unified process space. When a user executed a resource-intensive application on the master node, the SSI kernel would transparently migrate the process to the CPU of an idle worker node over the network.[52] The application itself required no modification; the kernel handled the forwarding of system calls, file descriptors, and memory state.[51]

While conceptually elegant, in the context of modern (2025/2026) Linux kernel architectures and contemporary hardware, the SSI paradigm is effectively obsolete and unsuitable for production environments.[28]

The death of SSI is attributed to several insurmountable architectural roadblocks. First, the performance costs associated with hardware privilege separation and security domains have escalated significantly.[28] Modern CPUs rely heavily on Translation Lookaside Buffers (TLBs) and strict memory isolation to prevent exploits like Spectre and Meltdown. The overhead of synchronizing these memory protections across a network connection destroys the performance benefits of distributed processing.[28] Furthermore, modern applications are heavily multi-threaded and utilize shared memory architectures (like NUMA) that are notoriously difficult to distribute transparently across physical network links without massive latency penalties.[55] While experimental academic projects like Popcorn Linux continue to research this domain [57], deploying kernel-level process migration in a custom distribution is practically unviable.

Evaluating Modern Distributed Paradigms: HPC vs. HTC

Modern distributed computing relies entirely on user-space scheduling, task orchestration, and

explicit parallelization rather than kernel-level illusion. To fulfill the user's requirements, the orchestration software must be selected based on the distinction between High-Performance Computing (HPC) and High-Throughput Computing (HTC).

1. **Slurm Workload Manager (HPC):** Slurm is the industry standard for tightly coupled supercomputer clusters executing MPI (Message Passing Interface) workloads.[59] It is highly efficient for massive parallel tasks. However, Slurm strictly requires a highly synchronized environment: identical user accounts across all machines, identical software environments, and a shared Network File System (NFS) to distribute data.[61] This makes it exceptionally fragile in an ad-hoc home or office LAN where older laptops and desktops may drop on and off the network unpredictably.
2. **Incus / Kubernetes (Container Clustering):** Systems like Incus or Kubernetes distribute workloads by orchestrating containers across nodes.[63] While highly scalable and fault-tolerant, these platforms are designed primarily for hosting microservices, databases, and persistent web daemons rather than dynamically scavenging idle CPU cycles for raw, temporary computational tasks.
3. **Distcc / Icecream (Distributed Compilation):** If the user's primary computational workload involves compiling software (e.g., building large C/C++ projects or custom kernels), tools like distcc and icecc (Icecream) are highly specialized solutions.[65] They intercept compiler commands and distribute the compilation of individual source files across the LAN.[67] Icecream is particularly suited for ad-hoc networks as it features a central scheduler that dynamically tracks which nodes are available and distributes work accordingly.[68] However, these tools are entirely restricted to software compilation and cannot pool CPUs for general-purpose data analysis or algorithmic tasks.
4. **HTCondor (HTC Cycle Scavenging):** Developed specifically for "cycle scavenging," HTCondor excels in heterogeneous environments with distributed ownership.[70] It is uniquely suited for processing massive backlogs of independent data tasks. HTCondor does not require a shared filesystem (NFS); it possesses a native file transfer mechanism that automatically pushes the input data to the worker node, executes the task, and pulls the output back to the master node upon completion.[70] Furthermore, it gracefully handles intermittent node failures—if a worker machine is turned off by a user or disconnected from the router, HTCondor detects the dropped connection, automatically requeues the task, and re-assigns it to another available node without any intervention from the master.[70]

For the user's specific requirement—adding varying, potentially older computers connected via a LAN router/switch to act as an additional CPU pool for a master laptop—**HTCondor** is unequivocally the superior architectural choice.[70]

Feature Metric	Slurm (HPC)	HTCondor (HTC	Icecream (Distributed

		Cycle Scavenging)	Build)
Primary Use Case	Tightly coupled parallel algorithms (MPI)	Batch data processing, independent task queues	C/C++ Source Code Compilation
File System Requirement	Strictly requires Shared Storage (NFS/Gluster)	Operates seamlessly without shared storage	Source files synced over network automatically
Node Availability	Assumes 100% dedicated uptime	Designed for intermittent availability (laptops/desktops)	Designed for intermittent availability
Resource Matching	Rigid Queue/Partition system	Dynamic "ClassAd" matchmaking	Dynamic CPU load balancing
Failure Handling	Fails job if node crashes	Automatically restarts/migrates job upon failure	Reassigns failed compilation to another node

Architecting the HTCondor Cluster Topology

Integrating HTCondor into the custom Debian live-build requires defining the specific roles of the machines within the ecosystem. The HTCondor architecture relies on several distinct daemons communicating over the LAN:

1. **Central Manager (condor_collector, condor_negotiator):** This is the brain of the cluster. It maintains the directory of all available resources on the network and matches incoming computational jobs to available worker nodes based on specific rules.[73] In this topology, the master laptop will run the Central Manager.

2. **Access Point / Submit Node (condor_schedd):** The interface where the user submits their computational jobs to the queue. It holds the jobs in a local spool until the Central Manager finds a suitable execution point.[73] This will also reside on the master laptop.
3. **Execution Point / Worker Node (condor_startd):** The daemon running on the background LAN computers. It receives jobs, executes them in an isolated sandbox, monitors resource consumption, and returns the results.[73]

HTCondor utilizes a highly expressive concept known as "ClassAds"—a matchmaking framework functioning similarly to a newspaper classified ad system.[70] When a worker computer on the LAN boots up and runs the HTCondor Execution Point daemon, it continuously publishes a ClassAd to the master laptop's condor_collector. This ClassAd details its specific hardware attributes, current CPU load average, available RAM, and operating system version.[70]

Concurrently, when the user wishes to utilize the cluster, they submit a job via the laptop using a Submit Description File. This file is effectively a reverse ClassAd representing the job's requirements (e.g., RequestCpus = 4, RequestMemory = 8GB, Requirements = (OpSys == "LINUX")).[59]

The condor_negotiator on the master laptop evaluates the pool of worker ClassAds against the submitted job ClassAds. If the requirements align, it establishes a direct connection between the submit node and the worker node. The input files and executable scripts are securely transmitted over the LAN via HTCondor's file transfer mechanism. The CPU computation executes on the worker, and upon completion, the output files and error logs are automatically compressed and returned to the master laptop's working directory.[70]

Implementation Strategy and Deployment Synthesis

The creation of this highly specialized, multi-disciplinary Linux distribution requires a rigorous, systems-level approach that harmonizes competing operational paradigms into a single, cohesive ISO image. By utilizing Debian's live-build infrastructure, the entire operating system is codified, ensuring reproducible and modular deployments across various hardware targets.[1]

To bridge the gap between stable server infrastructure and bleeding-edge penetration testing, the architecture vehemently avoids the "FrankenDebian" anti-pattern.[17] By strictly isolating Kali Linux and Parrot OS repositories via DEB822 cryptographic keyrings and enforcing APT Pinning policies—or optimally, by deploying Bedrock Linux strata via live-build hijack scripts—the core OS remains untouched. This guarantees that the user can seamlessly fetch and install tools from any security distribution without triggering catastrophic dependency conflicts.[25]

Hardware optimization for the master laptop requires a departure from legacy power configurations. The implementation of CONFIG_PREEMPT_DYNAMIC provides the critical flexibility to alternate between high-throughput server scheduling and low-latency interactive desktop modes at boot time.[37] Simultaneously, the integration of thermald and

hardware-based P-states (intel_pstate/amd_pstate) ensures the laptop can handle sustained computational cluster loads without succumbing to the severe performance degradation caused by passive thermal throttling.[43] Network bottlenecks are mitigated by expanding ring buffers and preparing the kernel for zero-copy XDP packet processing.[47]

Finally, the realization of a decentralized CPU resource pool abandons the obsolete concept of Single System Image kernel manipulation in favor of High-Throughput Computing principles.[28] By embedding HTCondor directly into the distribution's fabric, the master laptop gains the ability to seamlessly harvest idle CPU cycles from any connected machine on the LAN.

To fulfill the user's request of easily setting up different computers on the network, the live-build configuration can be engineered to generate a dual-purpose ISO. When booted normally, it launches the full desktop, custom kernel, and HTCondor Central Manager. However, a custom GRUB boot entry can be added during the live-build process labeled "Worker Node Mode." When the user flashes this ISO to USB drives and plugs them into various old desktops or laptops connected to the switch, selecting this mode strips away the graphical interface, loads into RAM, and automatically launches the condor_startd daemon configured to auto-discover the master laptop's IP address. This creates a zero-configuration deployment experience: the machines boot, advertise their CPUs via ClassAds, and instantly become an anonymous extension of the master node's resource pool. This architectural synthesis produces an environment uniquely suited for advanced security research, local server hosting, and scalable distributed computation.

Works cited

1. How to Make A Debian Base Linux Distro? - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/debian/comments/1r2mrj8/how_to_make_a_debian_base_linux_distro/
2. How to Create Custom Debian Based ISO - DEV Community, accessed on May 5, 2026, https://dev.to/vaiolabs_io/how-to-create-custom-debian-based-iso-4g37
3. Build a custom Debian live ISO - The Pragmatic Addict, accessed on May 5, 2026, https://pragmaticaddict.com/debian-live.html
4. Debootstrap - Debian Wiki, accessed on May 5, 2026, https://wiki.debian.org/Debootstrap
5. Build your own Customized Live Debian Distro using Debootstrap | Vasu's Va-super's Blog, accessed on May 5, 2026, https://www.vasuper.net/posts/debian-image-debootstrap/
6. accessed on May 5, 2026, https://www.linuxjournal.com/content/how-build-custom-distributions-scratch
7. Debian Live Manual, accessed on May 5, 2026, https://live-team.pages.debian.net/live-manual/html/live-manual.en.html
8. Building a custom Debian ISO image, accessed on May 5, 2026, https://debian-live-config.readthedocs.io/en/latest/custom.html
9. customizing-package-installation - Debian Live Manual, accessed on May 5, 2026, https://live-team.pages.debian.net/live-manual/html/live-manual/customizing-pac

kage-installation.en.html
10. [HOWTO] Make your own Debian Live ISO with live-build (repos included), accessed on May 5, 2026, https://lists.debian.org/debian-user/2025/09/msg00495.html
11. customizing-contents - Debian Live Manual, accessed on May 5, 2026, https://live-team.pages.debian.net/live-manual/html/live-manual/customizing-contents.en.html
12. Kernel Compilation in Debian Linux | dev-guides, accessed on May 5, 2026, https://cs4118.github.io/dev-guides/debian-kernel-compilation.html
13. Re: Building an image with a custom kernel - Debian Mailing Lists, accessed on May 5, 2026, https://lists.debian.org/debian-live/2016/03/msg00018.html
14. Compiling Custom Kernels in Debian | R.I.Pienaar - Devco.Net, accessed on May 5, 2026, https://www.devco.net/archives/2009/07/04/compiling_custom_kernels_in_debian.php
15. manojsgowda2910/Custom-Debian-ISO-Project - GitHub, accessed on May 5, 2026, https://github.com/manojsgowda2910/Custom-Debian-ISO-Project
16. How to build live image from custom compiled kernel? (Live-build) - Unix & Linux Stack Exchange, accessed on May 5, 2026, https://unix.stackexchange.com/questions/445395/how-to-build-live-image-from-custom-compiled-kernel-live-build
17. Is there any way to add all apt repository of Debian based Linux distros on your Debian system?, accessed on May 5, 2026, https://unix.stackexchange.com/questions/713760/is-there-any-way-to-add-all-apt-repository-of-debian-based-linux-distros-on-your
18. DebianRepository/UseThirdParty - Debian Wiki, accessed on May 5, 2026, https://wiki.debian.org/DebianRepository/UseThirdParty
19. Advanced Package Management in Kali Linux, accessed on May 5, 2026, https://www.kali.org/blog/advanced-package-management-in-kali-linux/
20. debian - Installing Kali Repositories - Super User, accessed on May 5, 2026, https://superuser.com/questions/1371161/installing-kali-repositories
21. Use Kali Linux tools in Ubuntu and other Debian based OS in a convenient way., accessed on May 5, 2026, https://viny1ic.medium.com/use-kali-linux-tools-in-ubuntu-and-other-debian-based-os-in-a-convenient-way-ec238634b2d8
22. Using Debian Bookworm, and tried adding Kali Repositories - much chaos and work (with a little bit of confusion and hilarity) ensued... | Linux.org, accessed on May 5, 2026, https://www.linux.org/threads/using-debian-bookworm-and-tried-adding-kali-repositories-much-chaos-and-work-with-a-little-bit-of-confusion-and-hilarity-ensued.48147/
23. How to safely add the Kali repository to Mobian - PINE64 Forum, accessed on May 5, 2026, https://forum.pine64.org/showthread.php?tid=10649
24. SecureApt - Debian Wiki, accessed on May 5, 2026, https://wiki.debian.org/SecureApt

25. How to add a third-party repo. and key in Debian? - Unix & Linux Stack Exchange, accessed on May 5, 2026, https://unix.stackexchange.com/questions/332672/how-to-add-a-third-party-repo-and-key-in-debian
26. Add Kali Repositories | The Journey - Chapter 1: Setting up Debian, accessed on May 5, 2026, https://scarecrow.gitbook.io/cyber-security/chapter-1-setting-up-debian/add-kali-repositories
27. Bedrock Linux, accessed on May 5, 2026, https://bedrocklinux.org/
28. SSI (single system image) nowadays. : r/osdev - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/osdev/comments/1bmy0kl/ssi_single_system_image_nowadays/
29. Bedrock Linux Explained in 2026: Combine Arch, Debian & Ubuntu in One Linux System, accessed on May 5, 2026, https://www.youtube.com/watch?v=zFj7G_gs7l8
30. TIL about Bedrock. have any of you created any twisted Frankenstein monsters using it? : r/linux - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/linux/comments/vz9jhm/til_about_bedrock_have_any_of_you_created_any/
31. Bedrock Linux on a Live Environment : r/bedrocklinux - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/bedrocklinux/comments/1qkugk0/bedrock_linux_on_a_live_environment/
32. Bedrock Linux Userland - GitHub, accessed on May 5, 2026, https://github.com/bedrocklinux/bedrocklinux-userland
33. Bedrock Linux 0.7 Poki Installation Instructions, accessed on May 5, 2026, https://bedrocklinux.org/0.7/installation-instructions.html
34. Power Management Guide | Red Hat Enterprise Linux | 7, accessed on May 5, 2026, https://docs.redhat.com/en/documentation/Red_Hat_Enterprise_Linux/7/html-single/power_management_guide/index
35. Power Management Strategies - The Linux Kernel documentation, accessed on May 5, 2026, https://docs.kernel.org/admin-guide/pm/strategies.html
36. Why would anyone choose not to use the lowlatency kernel? - Unix & Linux Stack Exchange, accessed on May 5, 2026, https://unix.stackexchange.com/questions/553980/why-would-anyone-choose-not-to-use-the-lowlatency-kernel
37. Kernel configuration options to enable for a laptop based server? Any extra tips for a student are welcome :) : r/linuxquestions - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/linuxquestions/comments/183sh11/kernel_configuration_options_to_enable_for_a/
38. How to Configure Linux Kernel for Real-Time Applications | Deterministic Low Latency Performance - YouTube, accessed on May 5, 2026, https://www.youtube.com/watch?v=iqWLWagb5JM

39. config_preempt_dynamic - kernelconfig.io, accessed on May 5, 2026, https://www.kernelconfig.io/CONFIG_PREEMPT_DYNAMIC
40. Revisiting the kernel's preemption models (part 1) - LWN.net, accessed on May 5, 2026, https://lwn.net/Articles/945227/
41. CONFIG_PREEMPT_DYNAMIC=y? - Google Groups, accessed on May 5, 2026, https://groups.google.com/g/linux.debian.kernel/c/0NXrmc4czQM
42. How do I compile Fedora kernel with preemption? (PREEMPT, not PREEMPT_VOLUNTARY), accessed on May 5, 2026, https://discussion.fedoraproject.org/t/how-do-i-compile-fedora-kernel-with-preemption-preempt-not-preempt-voluntary/77650
43. Linux Thermal Daemon Monitors and Controls Temperature in Tablets, Laptops, accessed on May 5, 2026, https://www.linuxfoundation.org/blog/blog/linux-thermal-daemon-monitors-and-controls-temperature-in-tablets-laptops
44. Using Processor Performance P-States with Linux on Intel-based ThinkSystem Servers, accessed on May 5, 2026, https://lenovopress.lenovo.com/lp1946-using-processor-performance-p-states-with-linux-on-intel-based-servers
45. A VM tuning case study: Balancing power and performance on AMD processors, accessed on May 5, 2026, https://developers.redhat.com/articles/2025/08/26/vm-tuning-case-study-balancing-power-and-performance-amd-processors
46. Linux Laptop Optimizations - Github-Gist, accessed on May 5, 2026, https://gist.github.com/Unixten/4483ec6e435fe6d7e2cb6dfac3f8ccd0
47. Chapter 33. Tuning the network performance | Monitoring and managing system status and performance | Red Hat Enterprise Linux, accessed on May 5, 2026, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/monitoring_and_managing_system_status_and_performance/tuning-the-network-performance_monitoring-and-managing-system-status-and-performance
48. Unleash Your Cloud: A Practical Kernel Tuning Playbook for Engineers | by Daman Singh Malik | Medium, accessed on May 5, 2026, https://medium.com/@malikdaman24/unleash-your-cloud-a-practical-kernel-tuning-playbook-for-engineers-aa0018d981a7
49. Get started with XDP - Red Hat Developer, accessed on May 5, 2026, https://developers.redhat.com/blog/2021/04/01/get-started-with-xdp
50. Fast Userspace Networking for the Rest of Us - arXiv, accessed on May 5, 2026, https://arxiv.org/html/2502.09281v1
51. Single system image - Wikipedia, accessed on May 5, 2026, https://en.wikipedia.org/wiki/Single_system_image
52. Scalable Cluster Computing with MOSIX for LINUX 1 Introduction, accessed on May 5, 2026, http://www.cs.columbia.edu/~orenl/papers/mosix4linux.pdf
53. OpenSSI - Wikipedia, accessed on May 5, 2026, https://en.wikipedia.org/wiki/OpenSSI
54. Software that distributes the performance load over multiple computers? - Server Fault, accessed on May 5, 2026,

https://serverfault.com/questions/178910/software-that-distributes-the-performance-load-over-multiple-computers
55. Unified Distributed OS for Commodity Clusters, accessed on May 5, 2026, https://cs191w.stanford.edu/projects/Spring2025/Arjun___Vikram_.pdf
56. Current single system image solutions - virtualization - Server Fault, accessed on May 5, 2026, https://serverfault.com/questions/1059908/current-single-system-image-solutions
57. Welcome to Popcorn Linux's documentation! — Popcorn Linux documentation, accessed on May 5, 2026, https://popcornlinux-doc.readthedocs.io/
58. Single system image (clustering) solutions - Unix & Linux Stack Exchange, accessed on May 5, 2026, https://unix.stackexchange.com/questions/382933/single-system-image-clustering-solutions
59. Translating between Slurm, HTCondor, and Torque - NRAO Information, accessed on May 5, 2026, https://info.nrao.edu/computing/guide/cluster-processing/appendix/translating-between-torque-htcondor-and-slurm
60. Mixing HTC and HPC Workloads with HTCondor and Slurm - CERN, accessed on May 5, 2026, https://s3.cern.ch/inspire-prod-files-0/0c84cb0e9b4c1a056e84cd2305ce2af0
61. Setting Up a Cluster of Tiny PCs for Parallel Computing | Hacker News, accessed on May 5, 2026, https://news.ycombinator.com/item?id=46710042
62. Convert your workflow from Slurm to HTCondor - OSPool Documentation - OSG Portal, accessed on May 5, 2026, https://portal.osg-htc.org/documentation/htc_workloads/submitting_workloads/Slurm_to_HTCondor/
63. How to Set Up Incus Clustering on Ubuntu - OneUptime, accessed on May 5, 2026, https://oneuptime.com/blog/post/2026-03-02-setup-incus-clustering-ubuntu/view
64. About clustering - Incus documentation - Linux Containers, accessed on May 5, 2026, https://linuxcontainers.org/incus/docs/main/explanation/clustering/
65. The C/C++ Developer's Guide – Part 2: icecream - UL Solutions, accessed on May 5, 2026, https://www.ul.com/sis/blog/cc-developers-guide-part-2-icecream
66. distcc: a fast, free distributed C/C++ compiler, accessed on May 5, 2026, https://www.distcc.org/
67. Accelerate Your Large Builds Locally With Distcc | Hackaday, accessed on May 5, 2026, https://hackaday.com/2024/03/04/oc-accelerate-your-large-builds-locally-with-distcc/
68. People who use distributed builds, how do you handle many compilers? - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/cpp/comments/y7ykx7/people_who_use_distributed_builds_how_do_you/

69. [HOWTO] Set up icecream - LinuxQuestions.org, accessed on May 5, 2026, https://www.linuxquestions.org/questions/slackware-14/%5Bhowto%5D-set-up-icecream-4175647336/
70. HTCondor Overview, accessed on May 5, 2026, https://htcondor.org/htcondor/overview/
71. An Introduction to Using HTCondor 2013, accessed on May 5, 2026, https://research.cs.wisc.edu/htcondor/HTCondorWeek2013/presentations/MillerK_IntroTutorial.pdf
72. Using a Job Scheduler on a desktop PC with excess compute resources. : r/HPC - Reddit, accessed on May 5, 2026, https://www.reddit.com/r/HPC/comments/ovhiok/using_a_job_scheduler_on_a_desktop_pc_with_excess/
73. Introduction — HTCondor Manual 25.0.9 documentation - Read the Docs, accessed on May 5, 2026, https://htcondor.readthedocs.io/en/lts/admin-manual/introduction-admin-manual.html
74. Downloading and Installing — HTCondor Manual 25.8.2 documentation, accessed on May 5, 2026, https://htcondor.readthedocs.io/en/latest/getting-htcondor/
75. HTCondor User Tutorial - CERN Indico, accessed on May 5, 2026, https://indico.cern.ch/event/936993/contributions/4022073/attachments/2105538/3540926/2020-Koch-User-Tutorial.pdf